

ReflexDebug: Debugger-Grounded Self-Healing Code Agent

Generative AI Laboratory, Lab CA

Self-healing code with ReAct (Reason + Act) and Reflexion loops

Name	Batch	Roll number
Aditya Ravi	B2	16014223004
Dirshak Deepak Patro	B3	16014223033

1. Aim

To design and build a Python agent that takes a programming task written in plain English, writes code for it, runs that code in a safe sandbox, and keeps repairing it from execution feedback until it is correct. The agent follows the ReAct and Reflexion patterns and grounds every repair in debugger-level evidence rather than in the traceback alone.

2. Objectives

- Build a sandbox that runs untrusted, model-written code with a static safety gate and OS-level resource limits, and reports failures in structured form.
- Turn each task into an executable specification (example tests plus property-based tests) before any code is written, so that wrong answers are caught, not only crashes.
- Capture the local variables of the failing frame and the shrunk counterexample for every failure, and let the agent query the crashed frame the way a person uses a post-mortem debugger.
- Implement a ReAct loop with tool calling (probe, inspect, patch, rewrite, dispute, finish), a loop guard against oscillation, and Reflexion trials driven by verbal self-reflection.
- Keep a long-term memory of debugging lessons, retrieved by embedding similarity for new tasks.
- Measure, with experiments that can be reproduced, how much diagnostic information each feedback channel carries, how safe the sandbox is, and how the full agent compares with simpler baselines.

3. Theory

3.1 Why generated code needs a feedback loop

Large language models write plausible code quickly, but a single sample is often wrong on edge cases: empty inputs, boundaries, validation rules, operator precedence. People fix such code by running it, reading the error, forming a hypothesis and trying again. A self-healing agent automates that cycle. The quality of the loop depends almost entirely on the quality of the feedback: a model can only fix what the feedback lets it see.

3.2 ReAct: reasoning and acting

ReAct (Yao et al., 2023) interleaves free-text reasoning (Thought) with tool use (Action) and the result of that tool (Observation). The reasoning trace lets the model plan and update its beliefs; the actions let it gather facts from the environment instead of guessing. In ReflexDebug, actions are debugging operations on a live program, and observations are structured test and debugger output.

3.3 Reflexion: learning from failed trials

Reflexion (Shinn et al., 2023) adds a verbal reinforcement signal. After a failed attempt the model writes a short self-reflection about what went wrong and what to do differently. That reflection is placed into the context of the next attempt, which acts like an episodic memory without any weight updates. We use it at two time scales: short-term reflections between trials of one task, and distilled lessons stored across tasks.

3.4 Self-debugging with execution feedback

Self-Debugging (Chen et al., 2024) showed that models improve when shown execution results of their own code, and LDB (Zhong et al., 2024) showed that runtime values of intermediate variables help more than the final error. This project follows the second idea: every failure is reported with the variables of the frame where it happened, and the agent can evaluate new expressions inside that frame.

3.5 Property-based testing

Example tests check a handful of inputs. Property-based testing, introduced by QuickCheck (Claessen and Hughes, 2000) and implemented in Python by Hypothesis (MacIver et al., 2019), states a rule that must hold for all inputs (a round trip, an invariant, agreement with a brute-force oracle) and searches for a counterexample. When one is found it is shrunk to a minimal failing input, which is exactly the kind of evidence a debugger wants.

3.6 Sandboxing untrusted code

Model-written code must be treated as untrusted. Static analysis of the syntax tree can reject dangerous imports and known escape routes, but Python is too dynamic for static checks alone to be a security boundary. A second layer therefore runs the code in a separate, isolated interpreter process with CPU, memory, file-size and wall-clock limits.

4. Problem statement and what is new

The basic self-healing loop described in the lab brief (generate, execute, capture the traceback, regenerate) has three weaknesses. First, it only reacts to crashes: code that returns a wrong value raises nothing, so the loop cannot see the bug at all unless tests exist. Second, a traceback names the line where an exception surfaced, which is often not the line that is wrong. Third, regenerating the whole program each time tends to fix one thing and break another, and the loop can oscillate between the same wrong versions. ReflexDebug addresses each of these with a specific mechanism, summarised below.

Weakness of the basic loop	ReflexDebug mechanism
Only crashes are visible	Spec-first: example tests and Hypothesis properties are synthesised and validated against a stub before code exists
Traceback line is not the faulty line	Crash-frame locals, shrunk counterexamples and an <code>eval_at_crash</code> tool that evaluates expressions inside the failing frame
Whole-program rewrites	<code>apply_patch</code> edits (whitespace-tolerant matching, edits that would not parse are rejected); full rewrite only when the design is wrong
The spec itself can be wrong	An independent review before any code is written, and evidence-based triage of failing tests once concrete counterexamples exist
Oscillation between wrong versions	Loop guard on AST-normalised code hashes and repeated error signatures; forces a fresh reflection
A wrong test can never be challenged	<code>dispute_test</code> sends the test, the evidence and the task (never the code) to an independent arbiter
Nothing is remembered between tasks	Reflexion self-reflections within a task and embedded lessons across tasks

5. System architecture

Figure 1 shows the complete pipeline. A task first becomes a validated specification. Lessons from earlier tasks are retrieved, and the coder writes a draft. Every draft and every edit passes the safety gate and runs in the sandbox, and the ReAct debugger works from the structured observations until the tests pass, the step budget runs out or the loop guard ends the trial. A failed trial produces a self-reflection that seeds the next one. At the end, a reusable lesson is distilled into long-term memory.

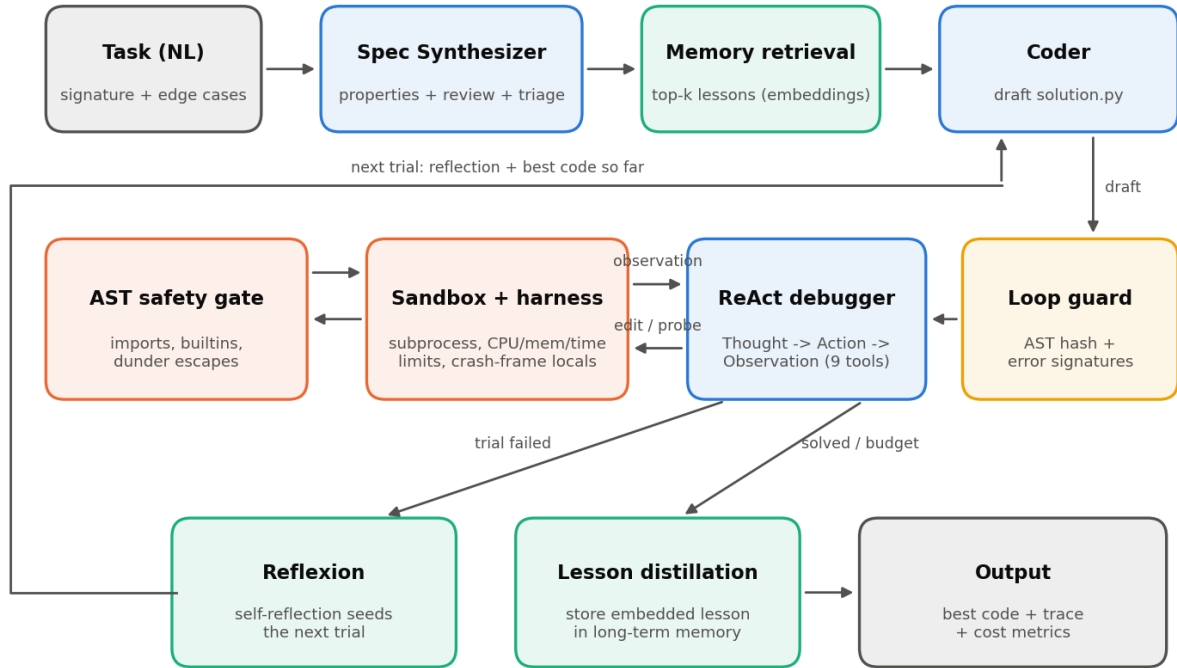


Figure 1. ReflexDebug architecture and control flow.

6. Algorithm

```

procedure REFLEXDEBUG(task T)
  L ← MEMORY.retrieve(T, k = 3)           # long-term lessons
  S ← SPEC_SYNTHESIZE(T)                  # examples + Hypothesis properties
  validate S against a stub; self-repair up to 2 times
  S ← SPEC_REVIEW(T, S)                   # drop / fix tests that contradict T
  (no valid S: return one unverified draft)
  R ← []                                  # short-term reflections
  for trial t = 1 .. MAX_TRIALS:
    code ← CODER(T, S, L, R, best_code)
    obs ← SANDBOX.run(code, S)             # gate, isolate, limit, crash-frame locals
    if t = 1 and obs has failures: S ← TRIAGE(T, S, counterexamples); re-run
    if obs.all_pass: break
    for step s = 1 .. MAX_STEPS:
      action(thought, args) ← LLM(REACT_PROMPT, history, TOOLS)
      obs ← EXECUTE(action)                # probe / inspect / patch / dispute ...
      obs ← obs + LOOP_GUARD(code, error_signatures, repeated_actions)
      if obs.all_pass or action = finish or guard.stuck: break
    best_code ← argmax over drafts of (all_pass, pass_rate)
    if not solved: R.append(REFLECT(T, trajectory, failures))
  lesson ← DISTIL(T, key events); MEMORY.store(lesson, embed(lesson))
  return best_code
  
```

The best draft is chosen by a simple score, pass rate plus one if every test passes, so a full pass always wins and a partially correct earlier draft is never lost when a later edit makes things worse. The same score decides which code is handed to the coder in the next trial.

7. Implementation

The system is written in Python 3 and uses the OpenAI Chat Completions API with function calling (default model gpt-4o-mini) and text-embedding-3-small for memory. It has no framework dependencies: the ReAct loop, the tool dispatcher and the memory store are implemented directly, which keeps every step visible and easy to test. Table 2 lists the modules.

Module	Responsibility
agent.py	Orchestrator: spec, Reflexion trials, ReAct loop, reflection, lesson distillation; emits an event stream consumed by the CLI and the web UI
spec.py	Spec synthesis (JSON mode), validation against a stub, vacuity check, self-repair, independent review pass, evidence-based triage of failing tests
tools.py	Nine ReAct tools as OpenAI function schemas (each with a required thought argument), whitespace-tolerant patching, and the observation formatter
sandbox.py	Temporary directory, isolated interpreter, empty environment, resource limits, timeouts
harness.py	Runs inside the sandbox: per-test alarms, Hypothesis profile, crash-frame capture, post-mortem evaluation, memory watchdog, pytest-style parametrize support
safety.py	AST gate: module allowlist, blocked builtins, dunder and frame attribute escapes
loopguard.py	AST-normalised code hashing, repeated error signatures and repeated identical actions
memory.py	Short-term reflections and long-term lessons with embedding retrieval
strategies.py	The four ablation arms behind a common interface
llm.py	OpenAI wrapper with retries, call budget, token and cost accounting; scripted offline model
prompts.py	Every prompt used by the system, in one auditable file

Table 2. Modules of the reflexdebug package.

7.1 Tools available to the agent

Tool	What it does
run_tests	Run the whole specification against the current code
inspect_failure	Full traceback, all captured locals and the falsifying example for one test
trace_test	Re-run a failing test under a line tracer: lines executed, locals that changed at each step, and the entry function's return value
eval_at_crash	Evaluate an expression inside solution.py where the exception was raised, or, for wrong-value failures, in the entry function's frame at its return
run_snippet	Execute a short probe with every top-level name of the solution available (including private helpers); like a REPL, the value of a trailing expression is shown
apply_patch	Replace a snippet that matches once (exactly, or ignoring indentation); edits that would not parse are refused; tests re-run automatically
rewrite_code	Replace the whole module when the approach itself is wrong
dispute_test	Ask an independent arbiter to keep, fix or remove a test that contradicts the task
finish	End the trial

Table 3. ReAct action space.

7.2 Specification synthesis and validation

The spec prompt asks for 5 to 9 example tests that cover edge cases stated or implied by the task, and 2 to 4 Hypothesis property tests with small, bounded strategies. The response is JSON with the required signatures and the test module. Before it is accepted, the test module must pass the safety gate, import cleanly, and run against a stub whose functions raise `NotImplementedError`. Errors that have nothing to do with the solution (`NameError`, bad Hypothesis strategies, wrong call signatures) are sent back to the model for repair, up to two times. Tests that pass against the stub are recorded as *vacuous*, because they cannot detect any bug.

7.3 Interfaces

`cli.py` renders the event stream in the terminal with coloured panels for thoughts, actions, observations, reflections and lessons, and prints a summary of steps, tool usage, tokens and cost. `app.py` is a Streamlit interface with a live trajectory, code diffs for each edit, a healing curve, tool usage, a memory browser and a benchmark dashboard. Both work in an offline demo mode that replays recorded model replies while executing every test for real.

8. Safety design

Two independent layers protect the host. Layer 1 parses every solution, test module, probe snippet and debugger expression and rejects it if it imports a module outside an allowlist (`os`, `sys`, `subprocess`, `socket`, `shutil`, `ctypes`, `importlib`, `pickle` and `urllib` are all outside it), calls `eval`, `exec`, `compile`, `open`, `__import__`, `input` or `breakpoint`, touches attributes used for sandbox escapes (`__subclasses__`, `__globals__`, `__mro__`, `__code__`, frame attributes and similar) or calls `getattr` with a computed or dunder name. Layer 2 runs whatever passes in a fresh temporary directory with a separate interpreter started in isolated mode (`-I`), an empty environment, its own session, and limits on CPU time, file size and core dumps. Each test has its own alarm inside a time budget for the whole run (tests that cannot be reached are reported as `NotRun`), and the process has a hard wall-clock timeout. macOS does not enforce address-space limits, so the harness also runs a memory watchdog thread that reads the kernel's physical-footprint counter, stops the process when it passes the limit and reports the exact line that was allocating.

We evaluated both layers with `benchmark/safety_eval.py`. The gate blocked 26 of 26 hostile samples and accepted 31 of 31 benign programs (all reference solutions, all hidden test modules and the demo files), so it produced no false positives on this corpus. Table 4 shows how the process layer handled programs that the gate allows but that abuse resources.

Program	Reported as	Time to contain
infinite loop	TestTimeout	3.28 s
memory bomb	MemoryLimitExceeded	0.41 s
deep recursion	RecursionError	0.23 s
output flood	MemoryLimitExceeded	0.60 s
exponential work	TestTimeout	3.24 s

Table 4. Resource-abuse programs contained by the sandbox.

9. Experimental setup

9.1 Benchmark tasks

We wrote 14 function-level tasks whose prompts are complete specifications but contain the kind of detail that single-shot generation tends to miss. Each task has a hidden test module that no agent ever sees, and a reference solution used only to prove that the hidden tests are correct.

Task	What makes it tricky
Semantic version comparison	pre-release ordering, numeric vs alphanumeric identifiers, strict validation
CSV line parser	quote state machine, doubled quotes, text after a closing quote
Arithmetic expression evaluator	precedence, right-associative power, $-2**2 = -4$, malformed input
Unix path normalisation	'..' at the root, '...' is a name, repeated slashes
Closed interval merging	touching vs adjacent intervals, unsorted input, no mutation
Strict Roman numeral parser	only canonical numerals are valid (IIII, VX, IC rejected)
ISBN-10 / ISBN-13 validator	X only in last position, ignored separators, never raises
Nested JSON flattening	list indices in paths, empty containers kept as leaves
Greedy word wrap	splitting over-long words while keeping the tail on the line
Strict base conversion	int() accepts '0x', '_', '+', and spaces; the spec forbids them
Deterministic topological order	lexicographically smallest order, cycle detection, duplicates
Strict duration parser	unit order, repeated units, unicode digits
Spiral matrix traversal	single row or column double counting, [[]] and ragged input
Run-length codec with bounded runs	runs longer than 9, digits as data, decode validation

Table 5. Benchmark tasks.

9.2 Ablation arms

Four strategies share the same model and the same budget of LLM calls per task. They differ only in the mechanisms listed in Table 6, so any gap between neighbouring arms can be attributed to the mechanism that was added.

Arm	Spec	Feedback	Repair	Reflexion + memory
One-shot	none	none	none	no
Traceback loop	3 to 5 plain asserts	raw traceback	full rewrite	no
Rich feedback	reviewed examples + properties	traceback, locals, falsifying example	full rewrite	no
ReflexDebug	reviewed examples + properties	same, plus debugger tools	ReAct tools, patches	yes

Table 6. Ablation arms.

9.3 Metrics

- Solve rate: share of tasks whose final code passes every hidden test.
- Hidden pass rate: share of hidden tests passed, averaged over tasks.
- False confidence: the agent's own tests all pass but the hidden tests do not.

- LLM calls, prompt and completion tokens, estimated cost in USD, and wall-clock time.
- For the offline experiments: mutation score, failure categories and availability of each feedback signal (Section 10.2), and block, accept and containment rates (Section 8).

10. Results

10.1 Verification of the infrastructure

The offline test suite (`tests/test_core.py`) passed 37 of 37 tests. It covers the safety gate, crash-frame capture, Hypothesis counterexamples, per-test timeouts, the memory watchdog, post-mortem evaluation, probing, patch matching, loop detection, memory retrieval and a complete scripted agent run. Every reference solution passed 100% of its hidden tests (99 test cases in total), which confirms that the hidden tests measure what the prompts ask for.

10.2 Mutation study: what does each feedback channel reveal?

A live LLM experiment tells us how well a particular model uses feedback. This experiment asks an earlier, model-independent question: when code is wrong, how much does each feedback channel actually reveal? We injected bugs into the 14 reference solutions with classic mutation operators (relational operator swap, arithmetic operator swap, integer constant off by one, and/or swap), keeping up to 24 mutants per task, and ran every mutant against the hidden tests in the real sandbox. For each detected mutant we recorded what a repair loop would be shown.

We generated 213 mutants and the hidden tests detected 199 of them, a mutation score of 93.4%. Figure 2 shows how the first failure of each detected mutant appeared, and Figure 3 shows how often each diagnostic signal was available.

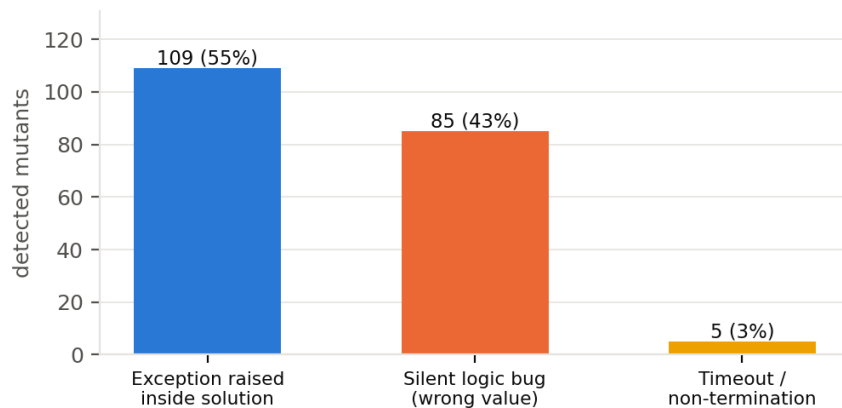


Figure 2. How the first failure of each detected mutant appeared.

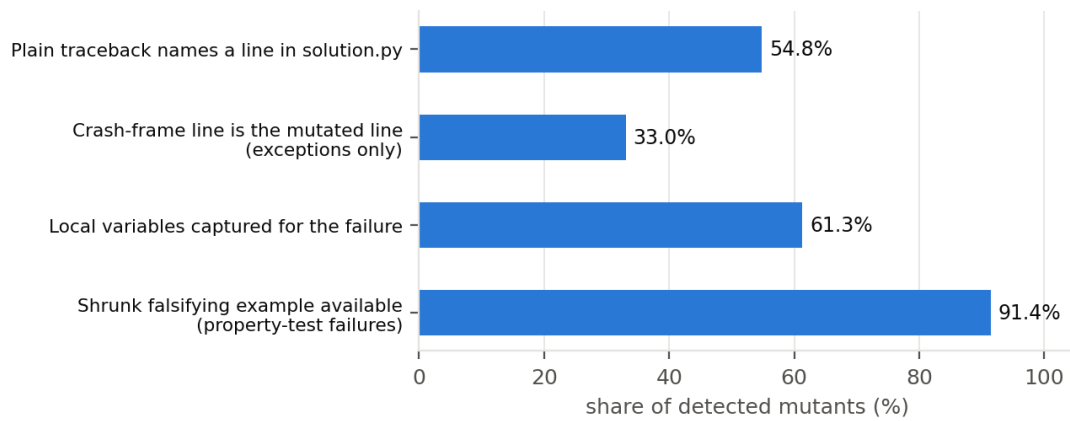


Figure 3. Availability of each diagnostic signal across detected mutants.

Task	Mutants	Detected
semver	24	20
csv_line	24	23
calc	24	23
path_norm	2	2
intervals	10	10
roman	22	20
isbn	24	24
flatten	1	1
wrap	9	7
base_conv	16	14
topo	8	8
duration	9	8
spiral	24	23
rle	16	16

Table 7. Mutants per task (tasks with little arithmetic or comparison logic yield fewer mutation sites).

10.3 Case study: one complete healing run

To show the loop end to end without an API key, the repository ships a scripted demonstration. The model replies are pre-recorded, but everything else is real: the specification is validated, every test run, debugger evaluation and probe executes in the sandbox, and the reflection and lesson go through the memory module. The task is a running median, where the even-length case must average the two middle values.

The first draft always took the upper middle element and passed 33% of the spec. Evidence-based triage kept the failing tests (they follow from the task). The agent then used `eval_at_crash` to read the state of `running_median()` at the moment it returned for the `shrunk` input:

```
Evaluated in solution.py running_median() at its return (line 7); the test failed with
AssertionError:
(s, n, result) = ([0, 1], 2, [0, 1])
```

It then patched the even-length branch, but kept floor division. The next observation shows the Hypothesis counterexample and the locals that exposed the remaining bug:

```

--- FAIL test_property_matches_oracle: AssertionError: got [0, 0], expected [0, 0.5]
Hypothesis Failing test case: test_property_matches_oracle(
  xs=[0, 1],
)
Test frame: spec_tests.py line 33: `assert got == expected, f"got {got!r}, expected
{expected!r}"`
  locals: xs=[0, 1], got=[0, 0], expected=[0, 0.5]
Traceback (most recent call last):
  File "spec_tests.py", line 30, in test_property_matches_oracle

```

The step budget ran out after a probe, so the trial ended and the agent wrote this reflection:

```

The root cause is in the averaging of the two middle values: floor division `//` truncates
1.5 to 1 and 0.5 to 0, so every even-length prefix with an odd sum is wrong. My patch
fixed the index selection but kept integer arithmetic, and I spent the last step probing
instead of fixing it. Next trial: compute the even case with true division `/ 2` and
keep odd-length prefixes returning the middle element unchanged.

```

The second trial, conditioned on that reflection, passed every test on its first draft, and the distilled lesson stored in long-term memory was: "When a result is the average of two integers (median, midpoint, mean), use true division `/`; floor division `//` silently truncates and only fails on odd sums." Figure 4 shows the pass rate after every observation. The run used 10 model calls in 2 trials.

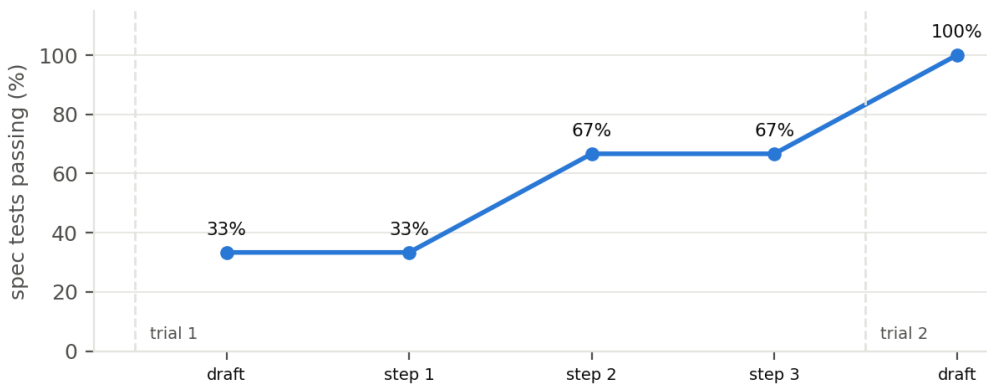


Figure 4. Specification pass rate across the scripted case study (real executions).

10.4 Pilot runs: what the live loop taught us

We did not tune the agent on the hidden tests. Instead we ran it live, read every step of its traces, and fixed what the traces showed was broken: first on two hard tasks (strict Roman numerals and the expression evaluator), then during a first benchmark run (v1) that we stopped after eight tasks once its traces exposed a design flaw, and finally from an audit of the complete second run (v2). Table 8 lists the 12 changes. They are the most direct evidence we have of how a self-healing loop behaves with a real model, and the v1 and v2 numbers are reported next to the final ones in Section 10.6.

Observed in the trace	What happened and what we changed
Near-miss patches	The model sent the same <code>apply_patch</code> five times in a row; it never matched because of indentation and a line-continuation backslash. Fix: whitespace-tolerant, line-level matching that re-indent the replacement.
Repeated actions went unnoticed	The loop guard only watched code states, so repeating a failed action was invisible to it. Fix: identical tool calls (same tool, same arguments) are now flagged with the result they produced the first time.
Reasoning disappeared	After a few turns gpt-4o-mini stopped writing free-text thoughts next to its tool calls. Fix: every tool schema has a required thought argument, so the ReAct reasoning trace is always present and logged.
Edits that break the syntax	Patching one line of a multi-line condition produced a <code>SyntaxError</code> and replaced working code. Fix: an edit that does not parse is refused with the surrounding lines shown, and the code is left unchanged.
The specification was wrong	One synthesised spec asserted that random letter strings are valid Roman numerals and called <code>from_roman</code> with an integer, so no correct solution could pass. Fix: a minimum number of single-behaviour tests and an independent review pass that fixes or removes tests that contradict the task.
Hangs blinded the whole run	In the first benchmark attempt a buggy CSV parser looped forever; every test hit its alarm, the run exceeded the wall-clock limit and was killed, so the agent saw no per-test evidence and one task took almost ten minutes. Fix: the harness tracks a time budget, shortens the last alarms to fit it, and reports tests it could not reach as <code>NotRun</code> instead of being killed blind. We restarted the benchmark from scratch after this fix.
The debugger was blind to wrong values	In v1 the agent spent 6 of 13 actions on <code>flatten</code> calling <code>eval_at_crash</code> for variables such as <code>current_path</code> and always got <code>NameError</code> : for an assertion failure the failing frame is the test, not the solution. This is the 42.7% silent-bug case from our own mutation study. Fix: an LDB-style line tracer. <code>trace_test</code> shows the executed lines with changed locals, and <code>eval_at_crash</code> now evaluates in the entry function's frame at its return.
Probes could not reach helpers	<code>run_snippet</code> imported the solution with a star import, which skips <code>_private</code> helpers, so probing the helper that contained the bug failed. Fix: all top-level names are exposed.
Reflexion trials repeated themselves	Trials 2 and 3 produced the same failing draft because the coder was shown the best code so far and copied it. Fix: later trials write a fresh implementation from the reflections at a higher temperature, and a draft identical to an earlier failed state is re-sampled.
Degenerate spec output	In v1 one spec call produced 16,384 tokens of repetitive text three times, took almost nine minutes and ended the run. Fix: an output cap on every call; a cut-off reply is rejected without being fed back, and the spec parser also accepts a test module given as a list of strings.
Wrong tests survived the review	The v2 audit (Section 10.7) found that 10 of 14 synthesised specs still contained at least one test that the reference solution fails, mostly property tests whose generators produce inputs the task excludes (start > end intervals, ragged matrices, empty keys). A review that only reads the tests cannot see this. Fix: evidence-based triage. After the first draft runs, the failing tests and their shrunk counterexamples go to the independent arbiter (without the code), which keeps, fixes or removes each one. The spec synthesiser also now fails cleanly instead of returning an invalid module.
Silent probes	The model called <code>evaluate('2 ** 3')</code> in <code>run_snippet</code> without <code>print</code> and read the empty output as a bug, wasting four steps. Fix: <code>run_snippet</code> behaves like a REPL and shows the value of a trailing expression.

Table 8. Design changes made after the live pilot runs.

10.5 Live ablation on the benchmark

All arms used model gpt-4o-mini with a budget of 20 LLM calls per task and 6 ReAct steps per Reflexion trial. Each arm ran once on each task. Table 9 summarises the four arms. One-shot generation solved 6 of 14 tasks with a mean hidden pass rate of 72.9%. The traceback loop, rich feedback and the full agent solved 5, 5 and 5 tasks, with mean hidden pass rates of 75.6%, 81.6% and 77.0%. The whole benchmark cost 0.20 USD.

Arm	Solved	Hidden pass	False conf.	LLM calls	Tokens	Cost (USD)	Time (s)
One-shot	6/14	72.9%	0	1.0	610	0.0032	4.5
Traceback loop	5/14	75.6%	2	7.21	7208	0.0300	31.6
Rich feedback	5/14	81.6%	2	11.14	16628	0.0633	83.8
ReflexDebug (full)	5/14	77.0%	2	13.5	40965	0.1046	66.8

Table 9. Ablation results on hidden tests (means per task, cost summed).

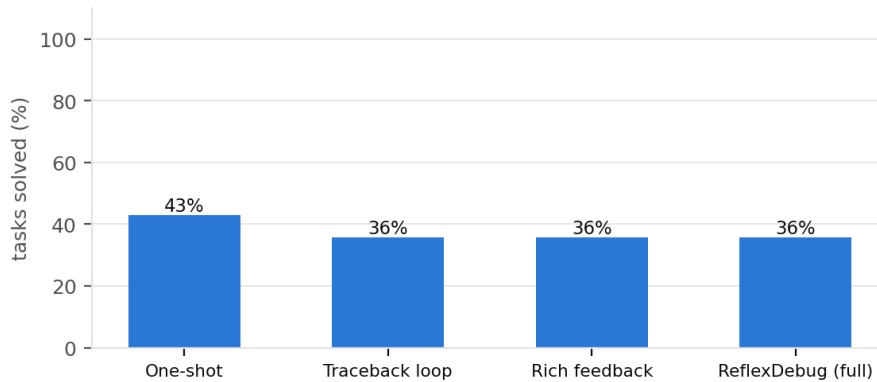


Figure 5. Share of tasks fully solved, by arm.

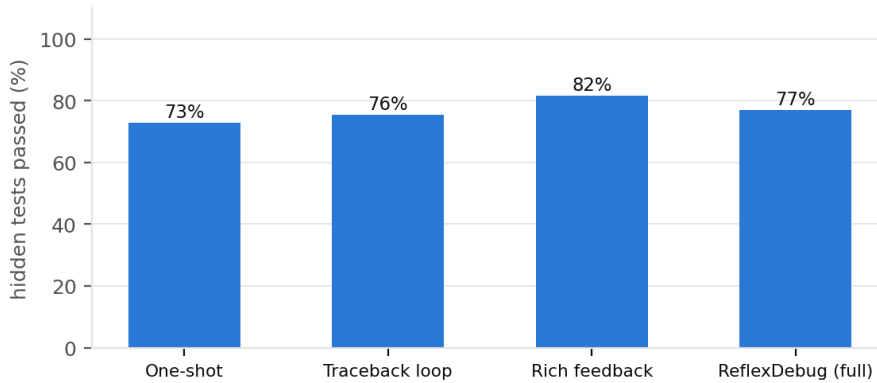


Figure 6. Mean hidden test pass rate, by arm.

Task	One-shot	Traceback loop	Rich feedback	ReflexDebug (full)
semver	93%	93%	80%	93%
csv_line	33%	67%	83%	67%
calc	0%	0%	0%	20%
path_norm	SOLVED	SOLVED	SOLVED	SOLVED
intervals	SOLVED	SOLVED	75%	SOLVED
roman	59%	71%	76%	71%
isbn	33%	33%	67%	67%
flatten	33%	33%	SOLVED	67%
wrap	75%	75%	75%	75%
base_conv	SOLVED	92%	92%	SOLVED
topo	SOLVED	SOLVED	SOLVED	25%
duration	93%	93%	93%	93%
spiral	SOLVED	SOLVED	SOLVED	SOLVED
rle	SOLVED	SOLVED	SOLVED	SOLVED

Table 10. Per-task result on hidden tests: SOLVED, or the share of hidden tests passed.

Agent behaviour

Across the 14 tasks the full agent ran 23 Reflexion trials in total (1.6 per task) and took 107 ReAct actions: 49 evidence actions (trace, eval, probe, inspect) and 58 edits. The spec review changed 12 synthesised tests before any code was written and evidence-based triage amended 27 more after the first draft ran. The loop guard fired 21 times, the agent itself disputed 0 tests, and 0 runs ended with an infrastructure error.

Tool	Calls	Share
apply_patch	57	53%
eval_at_crash	24	22%
inspect_failure	13	12%
trace_test	12	11%
rewrite_code	1	1%

Table 11. ReAct actions taken by the full agent (all tasks).

10.6 How the agent changed across versions

Table 12 shows the full agent in each development version: v1 (before the tracer, stopped after eight complete tasks), v2 (tracer and trace-driven fixes, before evidence-based spec triage) and the final version. The three baseline arms in Table 9 come from the v2 run and were not re-run, because none of the later changes touch them. Each version ran once, so differences of one task are within the run-to-run variation of the model.

ReflexDebug version	Tasks	Solved	Hidden pass	LLM calls
v1 (stopped)	8	2/8	56.5%	14.0
v2	14	5/14	78.2%	16.1
Final	14	5/14	76.9%	13.5

Table 12. The full agent across development versions.

10.7 Auditing the synthesised specifications

Because the reference solutions are known to be correct, each synthesised specification can itself be tested: a spec test that the reference solution fails is a wrong test. We also ran each run's first draft and final code against the hidden tests to see whether the repair loop helped (benchmark/spec_audit.py, no LLM calls).

Measure	v2 (review only)	Final (review + triage)
Runs with a spec	14	14
Specs the reference passes completely	4	7
Wrong spec tests (reference fails them)	18	13
Wrong spec tests before triage	18	17
Tests amended by triage	0	27
... of which the test really was wrong	0	14
First drafts that solve the task	4	5
Final code that solves the task	5	5
Mean hidden pass: first draft	73.4%	67.5%
Mean hidden pass: final code	78.2%	77.0%
Runs where repair improved hidden pass	1	4
Runs where repair lowered hidden pass	0	0

Table 13. Specification audit against the reference solutions.

11. Discussion

Silent bugs are common, and a traceback does not see them

Across the detected mutants, 42.7% showed up as a wrong value with no exception at all. For every one of those, the plain traceback pointed only at the test's assert statement and never mentioned solution.py. A loop that reacts to crashes alone cannot find these bugs, and a loop that shows only tracebacks gives the model no location to start from. This is why the specification is written first, why it includes property tests, and why the final agent traces wrong-value failures line by line.

The line in a traceback is usually not the faulty line

Even when an exception was raised inside the solution, the crash-frame line matched the mutated line in only 33.0% of cases. A wrong comparison upstream often surfaces as a ValueError or IndexError several lines later, so evidence tools are needed before patching.

Repair raised partial correctness, not the number of solved tasks

On the live benchmark every feedback loop raised the mean hidden pass rate above one-shot generation (72.9% for one-shot against 75.6%, 81.6% and 77.0% for the three loops), but none solved more tasks outright (6/14 for one-shot, 5/14, 5/14 and 5/14 for the loops). With gpt-4o-mini and 20 calls, the loops mostly turn half-right code into mostly-right code. Tasks that the model gets wrong conceptually (the expression evaluator, the strict ISBN rules) stayed wrong under every strategy.

The self-written specification is the bottleneck

The audit in Section 10.7 shows why. In the v2 run, 10 of 14 synthesised specifications contained at least one test that the correct reference solution fails. A loop that trusts such a spec can never accept correct code, so it keeps editing until the budget runs out: in v2 the full agent spent 16.1 calls per task on average even on tasks its first draft had already solved. Evidence-based triage raised the number of fully consistent specs from 4 to 7 of 14, cut the average to 13.5 calls and the total cost from 0.135 to 0.105 USD, and the share of runs in which repair improved the hidden result went from 1 of 14 to 4 of 14, with no run made worse in either version.

The arbiter is only as good as the model behind it

Triage has a price. Only 14 of the 27 tests it amended were really wrong; the rest were correct tests that gpt-4o-mini blamed for the code's failure, for example three valid example tests on the expression evaluator. Every wrongly removed test weakens the specification and can create false confidence. A stronger model as arbiter, or requiring several independent drafts to fail a test before it may be amended, are the obvious next steps.

Rich evidence helped most when paired with simple edits

The rich-feedback arm, which sees the same counterexamples and crash-frame locals but simply rewrites the whole module, reached the highest hidden pass rate at about half the cost of the full agent. The traces explain this: with the tracer the small model usually found the right evidence (for flatten the trace showed the empty list being skipped), but it often applied the fix in the wrong branch. For a small model, rich observations matter more than a large action space.

12. Limitations

- Each arm ran once per task with one model (gpt-4o-mini) and a budget of 20 calls. Model sampling is not deterministic, so differences of one task between arms are within run-to-run noise; several seeds and a stronger model would be needed for firm conclusions.
- The benchmark has 14 function-level tasks. It is designed to be tricky rather than large, so the results are a focused ablation, not a general leaderboard.
- The design was improved between runs using the traces of earlier runs (Section 10.4). We never looked at the hidden tests while doing this, but the final version was still shaped by experience on the same 14 tasks.
- The synthesised specification can be wrong, and the arbiter that corrects it is right only about half the time with this model. Hidden tests are used for evaluation precisely because self-verification is not trustworthy.
- The AST gate is a blocklist plus an allowlist, not a proof of safety; that is why the process layer exists. A production system should add a container or an OS sandbox profile with no network access.

- The memory watchdog samples every 20 ms, so a very fast allocation can briefly overshoot the limit. Output floods are held in memory and are stopped by the memory limit rather than by a dedicated output cap.
- Cost figures are estimates from published per-token prices.

13. Conclusion

ReflexDebug turns the basic generate, run, repair loop into an agent that writes its tests first, reads program state when something fails, edits surgically, notices when it is going in circles, reflects on failed trials and carries lessons forward. The offline experiments show why each mechanism exists: almost half of real bugs are silent, the traceback line is usually not the faulty line, and shrunk counterexamples are available for nearly every property failure. The sandbox blocked every hostile sample in our corpus without rejecting any legitimate code. The live experiments give a more sober picture than we expected. With a small model and a tight budget, feedback loops raise partial correctness but do not solve more tasks than a single sample, and the quality of the self-written specification decides how much the loop can achieve. Measuring that directly, by testing each specification against a known-correct solution, was the most useful thing we built: it explained the benchmark results, guided the triage mechanism, and showed exactly where the next improvement has to come from.

14. Future work

- A stronger model (or a panel of models) as the spec arbiter, and amending a test only when several independent drafts fail it.
- An adaptive controller that switches between whole-module rewrites and surgical patches depending on how localised the evidence is.
- Repeat the ablation with several seeds and a stronger model to separate method effects from sampling noise.
- Tree search over repair candidates (in the style of LATS) instead of a single trajectory per trial.
- Score synthesised specs automatically with the mutation harness, so weak specs are detected before repair.
- Run the sandbox in a container with no network namespace, and add an explicit output cap.

15. How to reproduce

```

pip install -r requirements.txt
cp .env.example .env          # add OPENAI_API_KEY
pytest                        # offline test suite
python cli.py check           # hidden tests vs reference solutions
python -m benchmark.mutation_study # Section 10.2
python -m benchmark.safety_eval  # Section 8
python cli.py demo             # Section 10.3, offline
python cli.py bench            # Section 10.5, needs a key
streamlit run app.py           # web interface
python report/build_report.py   # rebuild this PDF

```

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
- [2] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

- [3] X. Chen, M. Lin, N. Schaerli, D. Zhou. Teaching Large Language Models to Self-Debug. ICLR 2024.
- [4] L. Zhong, Z. Wang, J. Shang. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step (LDB). Findings of ACL 2024.
- [5] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, Y. Wang. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models (LATS). ICML 2024.
- [6] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J. Lou, W. Chen. CodeT: Code Generation with Generated Tests. ICLR 2023.
- [7] K. Claessen, J. Hughes. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. ICFP 2000.
- [8] D. R. MacIver, Z. Hatfield-Dodds, et al. Hypothesis: A new approach to property-based testing. Journal of Open Source Software 4(43), 2019.
- [9] Y. Jia, M. Harman. An Analysis and Survey of the Development of Mutation Testing. IEEE Transactions on Software Engineering 37(5), 2011.
- [10] OpenAI. Function calling and Embeddings API documentation. platform.openai.com/docs, accessed 2026.